

● LEXY

ENGINEERING DOCUMENTATION

DOCUMENT 04 / 09

System Design & Key Decisions

AI Hiring Platform · Intelligence-First

Generated **June 14, 2026** · Confidential — internal use

Lexy — System Design & Key Decisions (the “Why”)

Engineering documentation set · Doc 4 of 9 · **The most important document.**

Every team writes down *what* the architecture is. Almost no team writes down *why* it is that way — so every new engineer re-derives the reasoning by breaking things. This document is the reasoning. When you are tempted to “simplify” something here, read the **Why** first; most of these shapes are scar tissue from a real failure.

Each decision below follows: **Decision** → **Why** → **How to apply** / **what breaks if you don’t**.

■ Part A — The Intelligence Engine

A1. One composite score per (*candidate, job*), stored — not computed on read

Decision. Scores live in a `candidate_job_intelligence` table: five AI-computed dimensions (**Fit, Quality, Trust, Conversion** → **Hire Probability**) plus a `next_best_action` enum. `computeScores()` aggregates signals from the 10 agents; `upsertIntelligence()` persists.

Why. Scoring is expensive (multiple LLM calls + signal joins) and the same score is read on many surfaces (kanban, candidate card, decision queue, KPI cards, exports). Recomputing per request would be slow and *non-deterministic* — two surfaces could show different numbers for the same candidate in the same second. Persisting makes the score a **single source of truth** that every surface reads identically.

How to apply. The accrued intelligence row **wins on every surface**. Point-in-time match scores (e.g. `applications.match_score`) are *fallbacks* only. When building a per-candidate score map, take **MAX**, not last-write-wins — a later, lower point-in-time score must not overwrite a higher accrued one. (*memory: candidate-score-source-of-truth, kpi-list-count-consistency*)

A2. The four dimensions are weighted and deliberately overlap-free

Decision. Fit (skills/experience/ICP alignment), Quality (screening + interview performance), Trust (identity verification, proctoring, fraud risk), Conversion (engagement, ghosting resistance). Hire Probability is derived from these.

Why. Splitting into orthogonal axes lets the *recruiter see the reason*, not just a number. “Great fit, low trust” and “weak fit, high conversion” are different problems requiring different actions. A single black-box score can’t be acted on or defended.

How to apply. When you add a new signal, decide which dimension it belongs to and feed it there — don’t invent a new top-level number. Keep the axes interpretable.

A3. Confidence is a first-class output, and missing critical signals hard-cap it

Decision. Every score carries a `ConfidenceBreakdown` (completeness, freshness, critical-coverage). If a **critical** signal is missing (e.g. no screening or no interview yet), total confidence is **hard-capped** ($\approx 50\text{--}80\%$) no matter how good the partial data looks.

Why. A candidate with only a resume can look like a 95% match — and then bomb the interview. Reporting “95%, high confidence” off a resume alone trains recruiters to trust the number and then burns that trust. The cap makes the model *honest about what it doesn’t know yet*. Honesty about uncertainty is the whole sales argument for a “defensible” product.

How to apply. Never surface a high-confidence score that lacks critical coverage. A null score renders as `–`, never `0%` (a real 0 and “unknown” are different facts). (*memory: lexy-frontend-query-conventions, interview-per-question-scoring*)

A4. Time-sensitive signals decay; stale data loses influence automatically

Decision. Engagement / ghosting-risk style signals use **exponential half-life decay** (e.g. $\sim 24\text{h}$ half-life for ghosting risk).

Why. “Replied enthusiastically 3 weeks ago” is not the same as “replied today.” Without decay, a stale positive signal keeps a dead lead ranked high forever. Decay encodes the reality that recency matters for conversion signals.

How to apply. New conversion/engagement signals should decay; identity/skill signals generally should not (a verified identity doesn’t get staler).

A5. Per-tenant learned scoring — but gated, backtested, and never throwing

Decision. `learned-scoring.ts` periodically trains scoring weights on a tenant’s *own* hire/reject outcomes. Learned weights override the live config **only** after (a) the tenant passes a minimum **sample gate** (default 25) and (b) the new weights **win a backtest** against the current config. `getEffectiveScoringConfig()` **never throws** — on any error it silently falls back to the built-in config.

Why.

- *Sample gate:* training on 3 hires produces noise that looks like signal. Below the gate the tenant uses the safe default; learning is opt-in by data volume.
- *Backtest gate:* an unvalidated weight change can *degrade* hiring quality silently. Promotion only happens if it improves F1/precision on held-out history — the model must *earn* its way into production.
- *Never-throw:* scoring is on the hot path for every candidate surface. A thrown error in weight resolution would take down the core product. Silent fallback to built-in weights means “worst case we score like day one,” never “worst case the page 500s.”

How to apply. Train only via `trainTenantWeights`. Don’t add a code path that makes scoring throw. Don’t lower the sample gate to “see learning sooner.” (*memory: learned-scoring*)

A6. Cross-tenant global prior — the network effect, without moving raw data

Decision. Thin-data tenants **shrink toward a platform-wide meta-prior** learned by aggregating *sufficient statistics* (sums, sums-of-squares — scalars per dimension) across all tenants. Raw candidate rows **never** cross a tenant boundary. `getMetaPrior()` returning null falls back to the built-in prior (permanent fallback).

Why. This is the **moat**: every hire anywhere makes new/small tenants smarter, so the product gets better the more it is used — a real network effect. But “share data across tenants” is also the fastest way to a catastrophic privacy breach. Pooling only *scalar sufficient statistics* gives the statistical benefit (a sane prior to shrink toward) while making it structurally impossible to reconstruct another tenant’s candidates. The federated/backtest eval gate means a prior only ships if it actually helps.

How to apply. When extending cross-tenant learning, pool **aggregates only, never rows**. Keep the null-prior → built-in fallback path intact. (*memory: cross-tenant-global-prior, similar-hire-backtest-baseline*)

A7. Cold-start fallbacks must be deterministic, never random

Decision. When there’s no learned/accrued score yet, fall back to a **deterministic signal-based heuristic** — never `Math.random`.

Why. A random fitScore makes the same candidate jump around between page loads, destroying trust and making bugs unreproducible. There was literally a `Math.random()` fallback that had to be ripped out. Determinism is a correctness property here, not a nicety. (*memory: talent-match-coldstart-score*)

■ Part B — Tenant Isolation (RLS)

B1. Isolation is enforced in PostgreSQL with Row-Level Security, not in app code

Decision. Key tables carry RLS policies keyed on GUCs (`app.current_tenant_id`, `app.allowed_tenant_ids`). `withTenantContext` middleware sets `SET ROLE lexy_app` and those GUCs per request; handlers read through an RLS-bound Drizzle proxy pulled from `AsyncLocalStorage`.

Why. Multi-tenant data leaks are existential for a hiring product (one client’s candidates visible to another = lawsuit + dead company). Relying on every developer to remember `WHERE tenant_id = ...` in every query is a when-not-if failure: one forgotten clause leaks data. RLS makes the **database** the last line of defense — even a buggy handler can’t return another tenant’s rows because the connection itself is scoped.

How to apply. Treat RLS as a *backstop, not a substitute*. Still scope in the handler (see B2). Read `lib/db/src/index.ts` to understand the request proxy before touching the data layer.

B2. Application code must *also* scope with `getAllowedTenantIds` (defense in depth)

Decision. Use `getAllowedTenantIds(user)` — the full **subtree** — everywhere tenant data is read or linked. Gate any FK-keyed route (`jobId` , `candidateId` supplied by the caller) against the allowed set **before** using it.

Why. RLS protects rows you *select*, but it can't protect you from logic errors like “link this candidate to a job in a tenant the caller can't see” or “feed a foreign candidate's data into an LLM prompt.” Some report functions also legitimately need to widen to a subtree for parent visibility — that has to be done explicitly and carefully. Header presence is **not** auth: resolve the caller via `getAuthUserId` + lookup and 401 a null user *before* any role check.

How to apply.

- Validate caller-supplied `jobId` / `candidateId` against `getAllowedTenantIds` **before** linking rows or building a prompt.
- Staff-only routes need an **explicit** `STAFF_ROLES` **allowlist** — `getAllowedTenantIds` is *not* a staff gate (candidates also have a non-null `tenantId`).
- `tps_insert` /push-to-client policies are `WITH CHECK(true)` , so those routes **must** self-gate (staff allowlist + `clientTenantId ∈ allowed subtree`). (*memory: outreach-tenant-scoping, tenant-scoped-foreign-ids, auth-header-presence-not-auth, staff-only-route-role-gate, push-to-client-and-tps-rls*)

B3. Shared platform pool vs. tenant candidates

Decision. `candidates` is a **shared global talent graph**; tenant linkage is via `sourced_candidates` / `applications` . Some fields (e.g. `activityStatus` , `lastActiveAt`) exist **only** on platform-pool rows; tenant rows lack them.

Why. A candidate can be relevant to many tenants (especially across an agency's clients), so duplicating them per tenant would fragment identity and break dedup, scoring, and the network effect. But a shared pool means every read path must consciously decide “platform bypass or tenant-scoped?” — get it wrong and you either leak the pool or hide a tenant's own candidates.

How to apply. Gate platform-pool bypass on an explicit `candidateDatabaseAccess` capability — never `pool === 'platform'` unconditionally. For tenant rows, derive “active” from `updatedAt` since `activityStatus` is absent. Job-linked candidates must still appear in the job's Candidates tab even though the platform-pool gate would otherwise hide them — gate the foreign `jobId` up front instead. (*memory: platform-pool-isolation, candidates-list-activitystatus, job-candidates-tab-vs-pipeline-board*)

B4. Candidate de-duplication: one row per (tenant, lower(email))

Decision. Enforce one candidate per normalized email per tenant. Recruiter paths prompt-and-merge (409 `email_match`); async paths dedup or catch the unique violation (23505). **Never 500, never duplicate.**

Why. Duplicate candidate records silently corrupt everything downstream — they split scores, deflate connection-strength (a “0” is usually a duplicate, not a real no-engagement), and produce “Unknown” cards. The DB unique constraint is the enforcement; app code must handle the collision gracefully on both interactive and background paths. (*memory: candidate-email-dedup, connection-strength-zero-is-honest*)

Why a zero is a red flag, not a result. Connection-strength is *event-driven*: it accrues from real engagement (replies, interviews, outreach touches) recorded against a single candidate row. When a person gets split across two duplicate rows, the engagement attaches to one row while you’re looking at the other — so the row you see reports 0 even though the interaction genuinely happened. A true “no engagement yet” candidate and a duplicate-split candidate look identical (both show 0), which is exactly why a 0 should prompt “do I have a duplicate?” before it’s trusted as a real outcome. Fix the duplicate (merge) rather than “explaining” the zero.

■ Part C — Agent Orchestration

C1. Ten specialized agents, one orchestrator, a fixed canonical order

Decision. Agents (`icp`, `sourcing`, `screening`, `verification`, `outreach`, `interview`, `scheduling`, `anti-ghosting`, `proctoring`, `analytics`) are coordinated by `orchestrator.ts`. The pipeline runs them in a fixed `AGENT_ORDER`: `icp → sourcing → screening → verification → outreach → interview → scheduling → anti-ghosting → proctoring → analytics`.

Why. Each agent is a single-responsibility unit with its own model and metadata, which makes them independently testable and replaceable. The canonical order encodes real data dependencies — you can’t screen before you’ve sourced, can’t outreach before you’ve verified. Centralizing the order in one map (rather than hard-coding sequencing at call sites) means the pipeline is configurable and the dependency graph lives in one place.

How to apply. Add a new stage by extending the registry + `AGENT_ORDER`, not by chaining calls ad hoc. Every run is recorded (`pipeline_runs`, `agent_run_history`) and asynchronously feeds the intelligence layer (`_feedIntelligence`) — keep that feedback edge intact or scores go stale.

C2. Three execution modes: synchronous, chained pipeline, and the async queue

Decision. `triggerAgent` (one agent, sync, for real-time recruiter actions and interview turns); `runPipeline` (chained, output→input); and the `ai_jobs` queue drained by a worker loop using `FOR UPDATE SKIP LOCKED`.

Why. Different work has different latency/scaling needs. An interview turn must be synchronous (the candidate is waiting). Sourcing a whole pipeline is long-running and must not block an HTTP request. The queue decouples long/non-blocking work and `SKIP LOCKED` lets

multiple workers drain it safely in parallel without double-processing — that’s the horizontal-scaling story.

How to apply. Don’t run long agent work inside a request handler — enqueue it. (*memory: ai-job-queue*)

C3. Job-queue dedupe is a DB index; `enqueueAiJob` can return null

Decision. Deduplication of queued jobs is a **partial unique index** in the database, not application logic. `enqueueAiJob` returns **null** on failure, and the idle worker loop must honor `POLL_MS`.

Why. App-level dedupe races under concurrency (two requests check “is it queued?” simultaneously and both enqueue). A DB unique index is the only race-proof dedupe. Returning null (instead of throwing) forces callers to consciously handle “couldn’t enqueue” rather than crash.

How to apply. Callers must check the null return. Don’t reimplement dedupe in code. (*memory: ai-job-queue*)

C4. Approval-required by default; autonomy is opt-in and governed

Decision. Agent results default to **approval-required** — e.g. the outreach agent’s default outcome is `messagesPending > 0` (drafts awaiting a recruiter), which is **success, not failure**. Only an explicit `autoSendOutreach` / autopilot flag produces `messagesQueued/Sent`. Recruiters can edit AI drafts in place before approving.

Why. Sending email *as a client’s brand* to real candidates without a human in the loop is a brand/legal risk. The safe default is “AI drafts, human approves.” Autonomy exists but must be explicitly switched on per tenant policy (`lib/policies.ts`), not assumed. Treating “pending” as a failure would make callers think the agent broke when it actually did the right thing.

How to apply. Treat `messagesPending > 0` as success. Guard draft-status transitions **inside** the UPDATE predicate to avoid TOCTOU double-sends. (*memory: outreach-agent-result-shape, outreach-draft-edit-before-approval, agent-gate-vs-manual-override*)

C5. An explicit recruiter action overrides an automatic agent gate

Decision. When an agent gate (e.g. “candidate not verified → can’t enter outreach”) conflicts with an explicit recruiter stage move, the **human override wins** — but the bypass is scoped to that one trusted call-site, keeping dedup and approval intact.

Why. Agents enforce sane defaults, but the recruiter is the accountable decision-maker. A gate that can’t be overridden becomes a cage and the recruiter loses trust in the tool. Scoping the bypass narrowly prevents it from becoming a backdoor around *all* gates. (*memory: agent-gate-vs-manual-override*)

■ Part D — AI Governance, Fairness & Safety (why these are non-negotiable)

D1. Every candidate-facing AI message goes through a governed entrypoint

Decision. A kill switch + tenant scoping + atomic-send rules wrap every AI message entrypoint. A static check (`check:no-adverse-writes`) **fails the build** if any code writes an adverse stage without going through governance.

Why. Once you have N message entrypoints, “remember to add the safety checks” fails at entrypoint N+1. Making it a *build-breaking lint* means an ungoverned path can’t ship even if a reviewer misses it. The kill switch is the “stop all AI sending right now” lever you’ll want during an incident.

How to apply. Route new AI messaging through the governed helper; never write an adverse stage directly. (*memory: ai-message-governance*)

D2. Fairness directive + PII redaction wrap every candidate scorer

Decision. A `FAIRNESS_DIRECTIVE` is injected into the system prompt and `redactPii` strips names/protected characteristics (“blind screening”) before the LLM sees a candidate — on **every** scorer.

Why. Biased hiring AI is both unethical and illegal (disparate impact). Doing this on *most* scorers isn’t enough — the one unwrapped scorer is the one that gets you sued. Note: **esbuild won’t catch a missing import**, so a forgotten wrapper silently falls back to an unprotected score-50 rather than erroring — a dangerous silent failure. That’s why fairness has its own test suite.

How to apply. Wrap new scorers with the directive + redaction; run `test:fairness`. Watch the redaction regex edge cases (a phone-number regex must spare date ranges). (*memory: fairness-bias-mitigation, work-auth-interview-capture*)

D3. Adverse-impact uses the 4/5ths rule on (candidate, job) units

Decision. Adverse-impact analysis treats the unit as **(candidate, job)**, not application rows; “furthest stage reached” = `max(stage, event level)` (`INTERVIEW_STARTED` counts as interviewed). A demographics firewall is enforced by `test:fairness`.

Why. Counting application rows double-counts re-applications and understates how far a candidate actually got, distorting the 4/5ths ratio. The firewall keeps demographic attributes structurally separated from scoring so they can be used to *measure* fairness without ever *influencing* a decision. (*memory: adverse-impact-four-fifths*)

D4. Explainability & audit are product features, not afterthoughts

Decision. Recruiter and AI actions are audit-logged (`/audit`, SOC2 trail); client-facing evaluation can be exported as a PDF that faithfully shows verification state (flagged ≠ pending) and never leaks raw internal risk internals.

Why. “Defensible hiring” (the north star) is only real if a decision can be reconstructed after the fact. The export is what a client shows *their* auditor; exposing raw internal risk scores there would both confuse and create liability. (*memory: client-facing-evaluation-export*)

■ Part E — Cross-cutting infrastructure decisions

E1. `validate()` strips unknown body keys (silent by design)

Any `req.body` field a handler reads **must** be in that route's Zod schema, or it is silently dropped before the handler runs. *Why*: whitelisting is the safe default for an API (no mass-assignment), but it means “I added a field and it's mysteriously undefined” is always “you forgot the schema.” (*memory: validate-strips-unknown-body-keys*)

E2. External providers go through an adapter registry, never direct imports

Sourcing providers (PDL, SERP, GitHub, internal DB) are behind `lib/sourcing-providers.ts`. `runProvider` **never throws and never hangs** — it races each run against a per-kind timeout (discovery 20s / enrichment 90s, env-overridable) and degrades a failed/hung provider to a clean `skipped` result. *Why*: one flaky external API must not take down a whole sourcing run; graceful degradation that only caught *throws* still let a hung socket block the entire `Promise.all` indefinitely, so the timeout is the real fix. Disable a provider via `SOURCING_DISABLED_PROVIDERS` with no call-site edits. (*memory: sourcing-provider-adapter-layer*)

E3. Interview real-time pipeline: TTS must always settle; mobile STT is server-side

`speakText` is awaited before the mic opens and **must always settle** (watchdog + abort-late-TTS) or the candidate freezes on Q1. On mobile the browser can't use Web Speech (the video recorder holds the mic), so mic-only segments are routed to the server `/transcribe`. *Why*: a hung TTS or unavailable mic silently bricks the single most important candidate moment (the live interview). These are both “looked fine in dev, froze in production” failures. (*memory: interview-tts-watchdog, mobile-interview-stt*)

E4. Azure speech: pooled accounts + admission control, breaker is per-account

`pickAccount` returns null on an empty pool (never enter the Azure branch on null); circuit breaker is keyed per-account[::format]; `admit()` runs **last** after validate/auth. *Why*: under concurrent interview load a single Azure account throttles; pooling + admission control keeps latency sane, and running admission last means we don't reject a request before we've authenticated it. (*memory: azure-pool-and-admission, stt-metrics-persistence*)

■ How to use this document

- Adding to the scoring engine? Re-read **Part A** — especially A1 (source of truth), A5/A6 (gates and the privacy boundary).
- Touching anything multi-tenant? **Part B** is mandatory; RLS is a backstop, not a substitute for scoping.

- Adding an agent or background work? **Part C** — use the registry, the queue, and the approval-default.
- Anything that sends a message or scores a person? **Part D** is non-negotiable and partly build-enforced.

The deepest, most current “why” lives in `.agents/memory/` topic files — this document is the curated, stable digest. When the two disagree, the memory files and the code are newer; update this doc.

Ownership & keeping this current. This document has a named owner: the **project/engineering lead** (today, the project owner) is responsible for its accuracy. The trigger that keeps it from drifting is a rule, not good intentions: **any PR that adds, removes, or changes a decision in Parts A-E must update the corresponding section here in the same PR** — treat the doc edit as part of the change, the same way you’d update a test. Reviewers should block a decision-changing PR that leaves this doc stale. If you can’t reach the owner, still make the edit and flag it for review; a slightly rough update beats a confidently wrong one.